

Desarrollo del Taller de Python por Parejas

Integrantes del grupo:

David Alexander Ochoa Macias

Iván Leonardo Gómez Estupiñán

Nombre del instructor:

Andrés Felipe Sandoval Higuera

Programa de formación:

Análisis y Desarrollo de Software

Ficha:

3321349

Centro Minero

Fecha:

25/05/2026

Tabla de contenido

1. Introducción	3
2. Ejercicios	4
2.2 Ejercicio 1 – Análisis de Calificaciones	4
2.3 Ejercicio 2 – Lista de Compras Interactiva	5
2.4 Ejercicio 3 – Agenda de Contactos	6
2.5 Ejercicio 4 – Conversor de Unidades	8
2.6 Ejercicio 5 – Sistema de Inventario	10
3. Conclusión.....	13

1. Introducción

En este trabajo se desarrollaron diferentes ejercicios en Python utilizando estructuras como listas, diccionarios, ciclos y funciones.

Cada ejercicio permitió poner en práctica operaciones básicas de programación, como el manejo de datos, validaciones y menús interactivos.

Además, se utilizaron funciones integradas de Python para organizar mejor el código y facilitar la interacción con el usuario mediante programas sencillos y funcionales.

Enlace de github: https://github.com/davidalexandrochoamac1-cmd/Taller_Python_Grupal.git

2. Ejercicios

2.1 Ejercicio 1

```
# =====  
# EJERCICIO 1  
# Analisis de Calificaciones  
# =====  
  
def analizar_calificaciones(calificaciones):  
    promedio = sum(calificaciones) / len(calificaciones)  
    nota_alta = max(calificaciones)  
    nota_baja = min(calificaciones)  
    return promedio, nota_alta, nota_baja  
  
notas = [4.5, 3.8, 5.0, 2.9, 4.2]  
resultado = analizar_calificaciones(notas)  
  
print("----- 📋 Lista de calificaciones-----")  
print("=" * 60)  
print(notas)  
print("=" * 60)  
  
print("El Promedio grupal es :", round(resultado[0], 2))  
print("-" * 60)  
print("La Nota mas alta es :", resultado[1])  
print("-" * 60)  
print("La Nota mas baja es :", resultado[2])
```

Se creó una función llamada `analizar_calificaciones` utilizando `def`.

La función recibe una lista de notas y calcula el promedio, la nota más alta y la más baja utilizando las funciones `sum()`, `len()`, `max()` y `min()`.

Los resultados se retornan en una tupla usando `return`.

Después, se creó una lista de calificaciones de ejemplo y se utilizó `print()` para mostrar los resultados en pantalla de forma organizada.

2.2 Ejercicio 2

```
# =====
# EJERCICIO 2
# Lista de Compras Interactiva
# =====

lista_compras = []

print("="*60)
print("CALIFICACION LISTAS █")
print("="*60)

while True:
    print("\n➕ 1. Agregar item a la lista")
    print("➖ 2. Eliminar item de la lista")
    print("👁 3. Ver la lista completa")
    print("🚪 4. Salir")

    print("\n * 50")
    opcion = input("👁 Seleccione una opcion: ")

    if opcion == "1":
        producto = input("Ingrese el producto: ")
        lista_compras.append(producto)
        print(f"✅ El Producto {producto} fue agregado correctamente ")
    elif opcion == "2":
        producto = input("Ingrese el producto a eliminar: ")

        if producto in lista_compras:
            lista_compras.remove(producto)
            print(f"🔴 El Producto {producto} fue eliminado correctamente.")
        else:
            print(f"❌ El producto {producto} no existe en la lista.")
    elif opcion == "3":
        print("\nLista de compras:")

        if len(lista_compras) == 0:
            print("La lista esta vacia.")
        else:
            for producto in lista_compras:
                print(producto)
    elif opcion == "4":
        print("Programa finalizado.")
        break
    else:
        print("Opcion invalida. INTENTE DE NUEVO")
```

Se creó una lista vacía llamada `lista_compras` para almacenar los productos ingresados por el usuario.

Luego, se utilizó un ciclo `while` para mostrar un menú interactivo con diferentes opciones.

El programa permite agregar productos usando `append()`, eliminar productos con `remove()` y mostrar todos los elementos almacenados en la lista.

También se utilizó una condición if para verificar si un producto existe antes de eliminarlo y len() para comprobar si la lista está vacía.

Finalmente, se utilizó print() para mostrar mensajes y resultados en pantalla de forma organizada.

2.3 Ejercicio 3

```
def agregar_contacto(agenda):  
    nombre = input("👤 Ingrese el nombre del contacto: ")  
    telefono = input("☎ Ingrese el numero de telefono: ")  
    agenda[nombre] = telefono  
    print("✅ Contacto agregado correctamente.")
```

Se creó una función llamada agregar_contacto que recibe un diccionario llamado agenda.

Luego, se utilizaron input() para pedir el nombre y el teléfono del contacto.

Los datos se guardaron en el diccionario y finalmente se mostró un mensaje confirmando que el contacto fue agregado correctamente.

```
def buscar_contacto(agenda):  
    nombre = input("🔍 Ingrese el nombre a buscar: ")  
  
    if nombre in agenda:  
        print("📞 Telefono de", nombre + ":", agenda[nombre])  
    else:  
        print("❌ El contacto no existe.")
```

Se creó una función llamada buscar_contacto que recibe el diccionario agenda.

Luego, se utilizó input() para pedir el nombre del contacto a buscar.

Después, se utilizó if para verificar si el contacto existe en la agenda y mostrar su número de teléfono.

En caso contrario, se mostró un mensaje indicando que el contacto no existe.

```
def mostrar_contactos(agenda):
    print("\n 📅 Lista de contactos:")

    if len(agenda) == 0:
        print(" 📭 La agenda esta vacia.")
    else:
        for nombre, telefono in agenda.items():
            print(" 🧑 ", nombre, " | 📞 ", telefono)
```

Se creó una función llamada `mostrar_contactos` que recibe el diccionario `agenda`.

Luego, se utilizó `len()` para verificar si la agenda está vacía.

Si existen contactos, se utilizó un ciclo `for` junto con `items()` para mostrar el nombre y teléfono de cada contacto guardado.

```
agenda = {}

while True:
    print("\n 📅 AGENDA DE CONTACTOS")
    print("1. ➕ Anadir un nuevo contacto")
    print("2. 🔍 Buscar el telefono de un contacto")
    print("3. 📄 Mostrar todos los contactos")
    print("4. 🚪 Salir")

    opcion = input(" 🗑 Seleccione una opcion: ")

    if opcion == "1":
        agregar_contacto(agenda)
    elif opcion == "2":
        buscar_contacto(agenda)
    elif opcion == "3":
        mostrar_contactos(agenda)
    elif opcion == "4":
        print(" 🏁 Programa finalizado.")
        break
    else:
        print(" ⚠ Opcion invalida.")
```

Se creó un diccionario vacío llamado `agenda` para almacenar los contactos.

Luego, se utilizó un ciclo `while` para mostrar un menú interactivo con diferentes opciones.

Dependiendo de la opción seleccionada, el programa permite agregar contactos, buscar teléfonos o mostrar todos los contactos guardados.

Finalmente, se utilizó `break` para finalizar el programa cuando el usuario selecciona la opción de salir.

2.4 Ejercicio 4

```
conversiones = {  
    "metros": {  
        "pies": 3.28  
    },  
    "kilometros": {  
        "millas": 0.62  
    },  
    "kilogramos": {  
        "libras": 2.20  
    }  
}
```

Se creó un diccionario llamado conversiones para almacenar diferentes tipos de conversiones de unidades.

Dentro del diccionario principal se agregaron otros diccionarios con las equivalencias entre metros y pies, kilómetros y millas, y kilogramos y libras.

```
def mostrar_conversiones_disponibles():  
    print("Unidades disponibles: metros, pies, kilometros, millas, kilogramos y libras.")
```

Se creó una función llamada mostrar_conversiones_disponibles para mostrar en pantalla las unidades disponibles del programa.

Finalmente, se utilizó print() para mostrar las diferentes opciones de conversión.

```
def convertir(cantidad, origen, destino):  
    if origen in conversiones:  
        if destino in conversiones[origen]:  
            return cantidad * conversiones[origen][destino]  
        return "La unidad de destino no existe en el diccionario."  
    return "La unidad de origen no existe en el diccionario."
```

Se creó una función llamada convertir que recibe la cantidad, la unidad de origen y la unidad de destino.

Luego, se utilizaron condiciones if para verificar si las unidades existen en el diccionario de conversiones.

Si la conversión existe, se realiza la operación y se retorna el resultado.

En caso contrario, se muestra un mensaje indicando que la unidad no existe.

```
mostrar_conversiones_disponibles()
print("")
cantidad = float(input("💻 Ingrese la cantidad: "))
origen = input("📍 Ingrese la unidad de origen: ").lower()
destino = input("📍 Ingrese la unidad de destino: ").lower()

resultado = convertir(cantidad, origen, destino)

print("✅ Resultado:", resultado)
```

Se llamó la función `mostrar_conversiones_disponibles()` para mostrar las unidades disponibles.

Luego, se utilizaron `input()` para solicitar la cantidad, la unidad de origen y la unidad de destino.

Después, se llamó la función `convertir()` para realizar la conversión y finalmente se utilizó `print()` para mostrar el resultado en pantalla.

2.5 Ejercicio 5

```
def agregar_producto():
    nombre = input("📦 Nombre del producto: ")
    precio = float(input("$ Precio del producto: "))
    cantidad = int(input("📦 Cantidad disponible: "))

    producto = {
        "nombre": nombre,
        "precio": precio,
        "cantidad": cantidad
    }

    inventario.append(producto)
    print("✅ Producto agregado correctamente.")
```

Se creó una función llamada `agregar_producto` para ingresar nuevos productos al inventario.

Luego, se utilizaron `input()` para solicitar el nombre, precio y cantidad del producto.

Después, los datos se guardaron en un diccionario y se agregaron a la lista `inventario` utilizando `append()`.

Finalmente, se mostró un mensaje confirmando que el producto fue agregado correctamente.

```
def realizar_venta():
    nombre = input("📦 Producto vendido: ")
    cantidad_vendida = int(input("📦 Cantidad vendida: "))

    for producto in inventario:
        if producto["nombre"].lower() == nombre.lower():
            if producto["cantidad"] >= cantidad_vendida:
                producto["cantidad"] = producto["cantidad"] - cantidad_vendida
                print("✅ Venta realizada correctamente.")
            else:
                print("⚠️ No hay suficiente cantidad disponible.")
            return

    print("❌ El producto no existe en el inventario.")
```

Se creó una función llamada `realizar_venta` para registrar ventas en el inventario.

Luego, se utilizaron `input()` y un ciclo `for` para buscar el producto y verificar la cantidad disponible.

Si hay suficientes unidades, se actualiza la cantidad del producto.

En caso contrario, se muestra un mensaje de error.

```
def mostrar_inventario():
    print("\n 📋 Inventario actual:")

    if len(inventario) == 0:
        print(" 🚫 El inventario esta vacio.")
    else:
        for producto in inventario:
            print("-----")
            print(" 📦 Nombre:", producto["nombre"])
            print(" 💰 Precio:", producto["precio"])
            print(" 📦 Cantidad:", producto["cantidad"])
```

Se creó una función llamada `mostrar_inventario` para mostrar los productos guardados en el inventario.

Luego, se utilizó `len()` para verificar si el inventario está vacío.

Si existen productos, se utilizó un ciclo `for` para mostrar el nombre, precio y cantidad de cada producto.

```

while True:
    print("\n 📦 SISTEMA DE INVENTARIO")
    print("1. ➕ Agregar producto")
    print("2. 📦 Realizar venta")
    print("3. 📦 Mostrar inventario")
    print("4. 🚪 Salir")

    opcion = input("👉 Seleccione una opcion: ")

    if opcion == "1":
        agregar_producto()
    elif opcion == "2":
        realizar_venta()
    elif opcion == "3":
        mostrar_inventario()
    elif opcion == "4":
        print("👏 Programa finalizado.")
        break
    else:
        print("⚠️ Opcion invalida.")

```

Se utilizó un ciclo while para mostrar un menú interactivo del sistema de inventario.

Dependiendo de la opción seleccionada, el programa permite agregar productos, realizar ventas o mostrar el inventario.

Finalmente, se utilizó break para finalizar el programa cuando el usuario selecciona la opción de salir.

3. Conclusión

Con el desarrollo de estos ejercicios fue posible comprender mejor el uso de funciones, listas y diccionarios en Python.

También se fortaleció el manejo de ciclos y condiciones para crear programas interactivos capaces de realizar diferentes tareas.

Finalmente, las actividades permitieron mejorar la lógica de programación y la organización del código de una forma práctica y entendible.